

A canonic-signed-digit coded genetic algorithm for designing finite impulse response digital filter

Shing-Tai Pan *

Department of Computer Science and Information Engineering, National University of Kaohsiung, No. 700, Kaohsiung University Rd., Nan Dis., Kaohsiung 811, Taiwan, ROC

ARTICLE INFO

Article history:

Available online 2 July 2009

Keywords:

Canonic signed digit (CSD)

Genetic algorithm

Finite-impulse filter (FIR)

ABSTRACT

In this paper, the genetic algorithm (GA) based on Canonic Signed Digit (CSD) code was used to find the optimum design of a finite impulse response digital filter (FIR). By using the characteristics of the CSD structure, the circuit was able to be simplified and also the calculation speed was raised to increase the hardware's efficiency. However, CSD structure cannot be guaranteed by a general GA after the evolution of chromosomes. Thus in this research an algorithm was proposed which the CSD structure can be maintained. A CSD coded GA was used to the evolution of chromosome to reduce the time wasted by trials and errors during the evolution and then to accelerate the training speed. In this paper, a new hybrid code for the filter coefficients was proposed to improve the precision of the coefficient of FIR. An example is shown in this paper to verify the efficiency of the proposed algorithm.

© 2009 Elsevier Inc. All rights reserved.

1. Introduction

Because of the characteristics of the canonic signed digit (CSD), in recent years they have been broadly applied on hardware circuit structure simplification research. Not only do the characteristics of the CSD are achieved in the reduction of adders/subtractors and shift registers, it may also accelerate the calculation process, to achieve the performance of hardware circuit simplification and the acceleration of run speed [1–5]. On the achievement filter hardware, in order for system simplification and to accelerate operation speed, the number of shift registers and adders used must be reduced. The numeral system of the CSD code may just improve the hardware complexity of a filter with its characteristics, to achieve system simplification and to accelerate the operation speed of the system. Therefore in these recent years, many researches had added the CSD code structure. CSD is used on the numeral system of a binary numeral system. Since the CSD characteristics are unable to express within the whole binary numeral system, therefore it can only be expressed with similar optimization methods. Due to the contribution of simplification to the system, it still has been broadly used in the filter hardware realization.

In addition, the genetic algorithm (GA) was proposed by John Holland in 1960 to solve the searching for excellence. The application of GA is as like the human evolution method undergoing an evolution process. Darwin's Theory of Evolution had pointed the principle of "Living things compete, nature selects, the fittest survive, the unfitted are termination." Thus the whole gene evolution process included the choice of better reproduction, crossover and mutation abilities. The created offspring genes are meant to be better whereas poorer genes are gradually to be terminated [13,14,21,22]. With this method the poorer chromosomes are terminated to reserve the better chromosomes, so that the evolution of each generation will

* Fax: +886 7 5919514.

E-mail address: stpan@nuk.edu.tw.

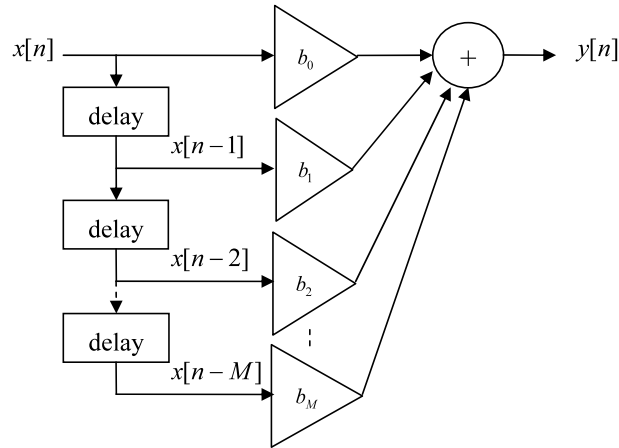


Fig. 1. The structure of a FIR.

be better and better. Thus, the genetic algorithm (GA) is a very good algorithm to achieve the optimum solution. Using the characteristics of natural human evolution, natural crossover and mutation, to create a new generation in which the gene performance of each offspring are better than the preceding generation, to reach the purpose of optimization and convergence performance [18,21–28].

Since a FIR with more coefficients of a higher order shall cause persecution to the design and carrying out process, and will result in complexity of the hardware structure, though a higher order causes more difficulty with design, the higher order relatively can obtain a better frequency response and a narrower frequency band transfer. Therefore if one wants to design a filter with good frequency response and narrow frequency band transfer, the order of the filter must be raised. Consequently, for the design of FIR filter, the CSD structure is used widely to reduce the complexity of the filter. There are many evolutionary methods for the design of the parameters of FIR, for example, Particle Swarm Optimization (PSO), Differential Evolution (DE) and Simulated Annealing (SE), etc. However, it is hard to match the CSD coded format into the chromosome of these algorithms during the evolution while GA is easy to do that. Therefore in recent years, much of GA and CSD research had included FIR, using GA to acquire the optimum algorithm. Also, in coordination with structure simplification of CSD, the achieving of FIR with higher orders is more convenient [6–9].

In past research, the GA based on the CSD code structure was studied for the design of filters, which there were two types of design methods. The first method was to run the inspection of the CSD structure after each generation of the GA calculating process. The second method was to switch all designed filter coefficients to the binary system code, and to finally transform the obtained binary system code into a CSD structure code. However the CSD structure code can be only a similar approach to the binary system code [1–4,6,8,15–17].

In this paper, GA was used to search for the optimum filter coefficient with the CSD code. Since the CSD code may be destructed during the GA calculating process, therefore in this paper a method was proposed to solve this issue. We proposed an evolution method, which completely followed the CSD rule during the calculation process and concurrently searched the optimum filter coefficient, maintaining the coefficient of the CSD framework.

The organization of this paper was as follows. Section 2 begins to introduce FIR. Section 3 gives an introduction about the coding of the coefficient of filter. Section 4 introduces the strategy of GA. In Section 5, the proposed GA was applied on the CSD code structure design and is the main key point in this paper. The method of how to use GA and maintain the CSD code structure are all introduced in this section. Section 6 shows the simulation results with the using of the CSD code structure.

2. Finite impulse response filter (FIR)

2.1. The finite impulse response filter module

A finite impulse response filter (FIR) is a non-recursive structural filter, which the framework is as shown in Fig. 1. The response equation of the corresponding structure is as [19]:

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] + \cdots + b_Mx[n-M]. \quad (2.1)$$

The above equation can also be written as

$$y[n] = \sum_{k=0}^M b_kx[n-k]. \quad (2.2)$$

If the input signal is an impulse signal function, $\delta(t)$, Eq. (2.1) can be rewritten as a pulse response equation shown below

$$h[n] = \sum_{k=0}^M b_k \delta[n-k]. \quad (2.3)$$

The z transformation of Eq. (2.1) is done by $Z\{x[n-k]\} = z^{-k}X(z)$ to obtain the following equation:

$$Y[z] = \sum_{k=0}^M b_k z^{-k} X[z]. \quad (2.4)$$

From Eq. (2.3) the transfer function $H(z)$ is acquired:

$$H(z) = \frac{Y(z)}{X(z)} = \sum_{k=0}^M b_k z^{-k} = b_0 + b_1 z^{-1} + b_2 z^{-2} + \dots + b_{M-1} z^{-(M-1)} + b_M z^{-M}. \quad (2.5)$$

Multiplying both sides of Eq. (2.5) with z^M can give the following transfer function:

$$H(z) = \frac{b_0 z^M + b_1 z^{M-1} + b_2 z^{M-2} + \dots + b_{M-1} z + b_M}{z^M}. \quad (2.6)$$

There are a number of $M+1$ coefficients in the FIR, which relatively has a M zero values and M poles. From Eq. (2.6) it can be realized that the poles in the FIR are 0, and therefore is an absolute stable system.

Let $z = e^{j\Omega}$, which the frequency response function can be expressed as

$$H(\Omega) = \sum_{k=0}^M b_k e^{-jk\Omega}. \quad (2.7)$$

2.2. Estimation of the order of a filter

Once the conditions of the to-be-designed filter were set, the order of the filter must be determined. The equation below is a order N_c estimation for the designing of a low pass filter [20],

$$N_c \approx \frac{D_\infty(\delta_p, \delta_s)}{\Delta F} - f(\delta_p, \delta_s) \Delta F + 1 \quad (2.8)$$

where N is the smallest integer which is greater than N_c and ΔF is the transfer frequency band based on a selected frequency of the filter.

$$D_\infty(\delta_p, \delta_s) = \log_{10} \delta_s [a_1 (\log_{10} \delta_p)^2 + a_2 \log_{10} \delta_p + a_3] + [a_4 (\log_{10} \delta_p)^2 + a_5 \log_{10} \delta_p + a_6], \quad (2.9)$$

$$f(\delta_p, \delta_s) = 11.01217 + 0.51244 [\log_{10} \delta_p - \log_{10} \delta_s], \quad (2.10)$$

$$\begin{aligned} a_1 &= 5.309 \times 10^{-3}, & a_2 &= 7.114 \times 10^{-2}, \\ a_3 &= -4.761 \times 10^{-1}, & a_4 &= -2.66 \times 10^{-3}, \\ a_5 &= -5.941 \times 10^{-1}, & a_6 &= -4.278 \times 10^{-1}. \end{aligned} \quad (2.11)$$

The sign δ_p is the magnitude of the ripple or the relative error with the pass band of an ideal filter of the pass band, and δ_s is the magnitude of the ripple or the relative error with the pass band of an ideal filter of the stop band.

From the above equations, one may not only design the required filter coefficient length, but also can use to design the frequency shift width once the coefficients of filter length are determined.

3. The code of the coefficient of filter

In this paper, the appropriate changes of the FIR coefficients were based on the structure of a power-of-two code [6,10–12] to produce the evolutionary hybrid code, and to create a new structural code. In this section the hybrid code method is first introduced, and furthermore the structure of the hybrid code in this paper is introduced. An example is also given to compare the difference of the two, in which gives the reason of why is a new hybrid code used.

3.1. The hybrid code method

The hybrid code is a proposed code method which improves the general sign binary code. The calculation method is quite similar with the general sign binary algorithm. For the general sign binary code calculations, the most significant bit (MSB) is a sign bit. A positive is represented when the sign bit equals 0, and negative when the value is 1. As for the calculation for the hybrid code, besides the calculation method is partially the same with the general binary code, the difference is that the acquiring values in the hybrid code are of its indicators. Since in an ordinary calculation the values obtained are positive integers, therefore in order to suit for the use of the following designed filter coefficients, the obtained values must then be taken in its reciprocal form. The structure of the hybrid code is shown in Fig. 2 [6]:

S_{k_1}	k_1	S_{k_2}	k_2	$\dots$	S_{k_m}	k_m
-----------	-------	-----------	-------	---------	-----------	-------

Fig. 2. Structure of hybrid code.

in which

$$k_i = \sum_{j=0}^n b_{ij} \times 2^j, \quad i \in \{1, 2, \dots, m\}, \quad (3.1)$$

and S_{k_i} , $i \in \{1, 2, \dots, m\}$, is the sign bit of k_i ; b_{ij} , $j = 0, 1, 2, \dots, n$, refers to the j th bit of k_i .

The calculation of the coefficients is as shown below:

$$\text{coefficient} = \sum_{i=1}^m S_{k_i} \times 2^{-k_i}. \quad (3.2)$$

For example, the number 0.125 can be expressed as follows:

$$\text{coefficient} \quad 0.125 = 2^{-2} - 2^{-3}$$

S_{k_1}	k_1	S_{k_2}	k_2
0	0 1 0	1	0 1 1

3.2. New hybrid code method

The calculations of the new hybrid code and original hybrid code have parts which are similar. The similar parts are the sign bit and the first part of the indicator calculations, which the sequential indicator calculations are different with the hybrid code. In the new hybrid code, the calculation is by accumulation of the previous indicators. Since by this method, the adjustment for even tinier numerical values to raise the numeral precision can still be done. The following is the calculation method for the new hybrid code.

Calculation of the indicating numbers:

$$a_i = \sum_{l=1}^i k_l. \quad (3.3)$$

Calculation of the coefficients:

$$C = \sum_{j=1}^m S_{k_j} \times 2^{-a_j}. \quad (3.4)$$

In (3.4), C represents the value of a coefficient, for example $\text{coefficient } 0.21875 = 2^{-2} - 2^{-5}$.

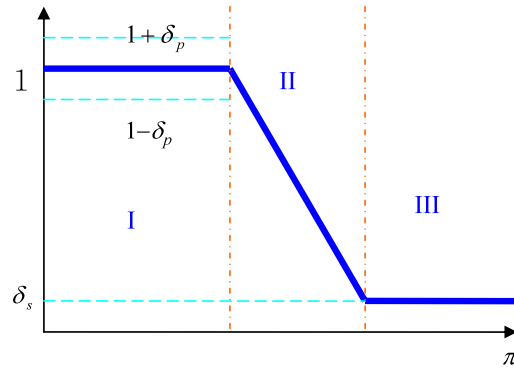


Fig. 3. The ideal frequency response and indication of δ_p and δ_s .

S_{k_1}	k_1	S_{k_2}	k_2
0	0 1 0	1	0 1 1
+	2	–	3
+	2^{-2}	–	$2^{-(2+3)} = 2^{-5}$

From the above example, the binary system code and the hybrid code are completely the same, however the results are different where though in k_2 the value is 3, it is required with to accumulate with the previously obtained k_1 value. Thus, the final results are completely different.

The purpose of this proposed method is to achieve a fine tuning operation during the calculating process. From the above example it is obvious to see that for the final values, only the value of k_1 is approaching. Therefore the following produced values can create a fine tuning effect for the main values in k_1 . Since the hybrid code method cannot perform fine tuning adjustment, therefore this new method is brought upon for the calculation of the coefficients. It is mainly used for the first part, and the following part finely tunes the coefficients to increase precision, which can easier obtain optimization.

4. Genetic algorithm

4.1. Definition of the fitness function

For an actual GA application, the fitness function is used to calculate the genes of each generation. As for adaption, different fitness functions are used for different environments. We use the fitness function to define the quality of each gene. The definition of the fitness function for the FIR design in this paper is as follows:

Let the transfer function of the FIR be as belows:

$$H(z) = \sum_{k=0}^N b_k z^{-k}, \quad (4.1)$$

where N represents the order of the transfer function $H(z)$.

Let $z = e^{j\Omega}$, and the magnitude frequency response equation of $H(z)$ is:

$$H(\Omega) = \sum_{k=0}^N b_k e^{-jk\Omega}. \quad (4.2)$$

And thus the mean square error (MSE) for the frequency response of the designed filter with an ideal filter is

$$E_p \equiv \frac{\sum_{\Omega} [H(\Omega) - H_I(\Omega)]^2}{N_s}, \quad 0 \leq \Omega \leq \pi, \quad (4.3)$$

where $H(\Omega)$ is the d frequency response magnitude, of the designed filter, $H_I(\Omega)$ is the frequency response magnitude of an ideal filter, and N_s is the sample collected points.

In the paper, the calculation for this part used the concept of offset (δ_p and δ_s), where δ_p is the pass band offset and δ_s is the stop band offset, as compared in Fig. 3. Within the designing of the filter, the ripple phenomena both exist for the pass band and stop band. Therefore offset is used to set the allowable ripple magnitude, which to control the frequency response of the filter.

The procedure to define the fitness function, by using the offset concept, in this paper is as follows:

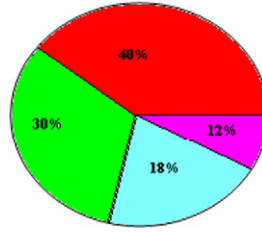


Fig. 4. The roulette wheel area distribution.

Step 1: At first consider zone I

When $1 + \delta_p \geq H(\Omega) \geq 1 - \delta_p$

$\Rightarrow error_I(\Omega) \equiv 0$;

If $H(\Omega) > 1 + \delta_p$ or $H(\Omega) < 1 - \delta_p$

$\Rightarrow error_I(\Omega) \equiv [H(\Omega) - H_1(\Omega)]^2$.

Step 2: Consideration of zone II

$error_{II}(\Omega) \equiv [H(\Omega) - H_1(\Omega)]^2$.

Step 3: Consideration of zone III

When $H(\Omega) \leq \delta_s$

$\Rightarrow error_{III}(\Omega) \equiv 0$;

If $H(\Omega) > \delta_s$

$\Rightarrow error_{III}(\Omega) \equiv [H(\Omega) - H_1(\Omega)]^2$.

Step 4: Calculate the error function

$$E_p \equiv \frac{\sum_{\Omega} [error_I(\Omega) + error_{II}(\Omega) + error_{III}(\Omega)]}{N_s} \quad (4.4)$$

in which E_p is the error function caused by p , N_s is the collected points between $0 \sim \pi$.

Step 5: Calculation of the fitness function

$$fitness(P) = \frac{1}{E_p}. \quad (4.5)$$

4.2. Chromosome selecting method

The roulette wheel method is used as the selecting method for the next generation of chromosomes. The roulette wheel distributes the proportion of area by the relative-adapting ability to the environment of each chromosome. The probability equation is as follows:

$$prob(P_i) = \frac{fitness(P_i)}{\sum_{i=1}^n fitness(P_i)}. \quad (4.6)$$

Better chromosomes with larger fitness will receive a relatively larger distributing area, and the chance of being selected increases. Also, a better chance there is to terminate the poorer chromosomes and to leave the better chromosomes. Fig. 4 shows an example of roulette wheel according to Eq. (4.6).

4.3. Crossover

The crossover method used in the paper is the double-point crossover. Assume two chromosomes, P_1 and P_2 , in which the gene length are both 8 bits. The crossover points are #2 and #6 (counting from the right, and are suitable to all the examples in the paper). The crossover method is as follows:

Before the crossover:

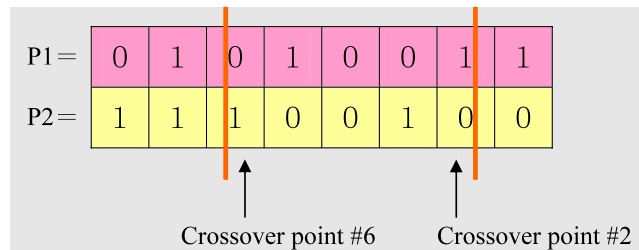
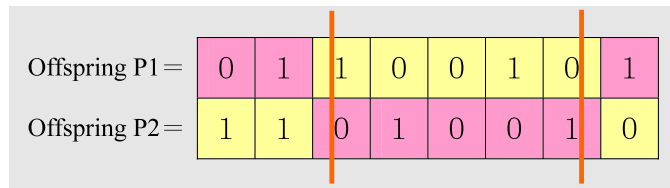


Table 1

Filter coefficients of using 2 as the complement number [1].

k	$h(k)$	$h(k)$ scaled	$h(k)$ rounded	$h(k)$ 16-bit, 2's complement	Total adds
0	−0.0057534026	−575.34026	−575	1111 1101 1100 0001	6
1	0.00099026691	99.026691	99	0000 0000 1100 0011	4
2	0.0075733471	757.33471	757	0000 0010 1111 0101	8
3	−0.0065141204	−651.4120	−651	1111 1101 0111 0101	10
4	0.013960509	1396.0509	1396	0000 0101 0111 0100	8

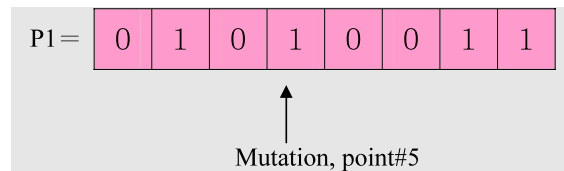
The double-point crossover method is by exchanging the values of the two points that is to crossover. The results after the crossover are as below:



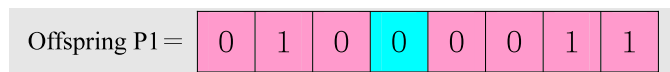
4.4. Mutation

Generally, the chance of mutation is set between 1%~8%. The mutation rate cannot be set too high, since with a higher mutation it will cause the original characteristics to completely vanish. This will cause the result of hardly to converge, and then unable to achieve an optimum solution. If the chance of mutation is exceedingly low, the convergence rate might be too slow, increasing search time. Thus an appropriate mutation enables to have a better efficiency. The mutation rate set in this paper is 1%. The following example is a simple mutation for example: Assume a chromosome with gene length of 8 bits, the mutation point is #5 as shown below:

Before the mutation



Mutation changes the original numerical value from 1 to 0, or from 0 to 1, as shown below:



5. CSD coded GA

5.1. CSD code

The rules of simplification of CSD to the system are in which two consecutive digits both cannot be of zero value. The product of every two consecutive digits must be zero, i.e. $b_n \times b_{n+1} = 0$. With this characteristic, as the system produces lesser bits of non-zero values, the system requires less shifting and adding operations. Therefore, in relative with the numerical values of the normalized binary system, the maximum number of non-zero bits in a CSD will be less or equal to half of that in a normalized binary system. Thus the function of simplification can be acquired to accelerate the operation speed. We use part of the data from reference [1] to compare the advantages. Table 1 is the comparison table when using 2 as a complement number for the code. Table 2 is the comparison table as expressed in the CSD code. From Tables 1 and 2, it can be apparently seen that the number of adder/subtractors are less with a CSD structure.

Since the CSD structure will be destructured in a general GA, thus in past research, the code was needed to be examined after each time a gene evolution process was finished to see whether the CSD structure remained or not. If it turned out not

Table 2

The filter expressed in the CSD code structure [1].

k	$h(k)$	$h(k)$ rounded	$h(k)$ canonical signed digit	Adds	Subtracts	Total
0	−0.0057534026	−575	0000 0010 0100 0001	1	2	3
1	0.00099026691	99	0000 0001 0100 0101	2	2	4
2	0.0075733471	757	0000 0101 0001 0101	3	2	5
3	−0.0065141204	−651	0000 0010 1001 0101	2	3	5
4	0.013960509	1396	0000 1010 1001 0100	2	3	5

being the CSD structure, adjustments were required. This kind of inspection method causes the search time for an optimum solution to increase, and some few errors are created when transforming the non-CSD structure code to a CSD structure. Therefore in this paper, in order to decrease the time of examining the CSD structure and to reduce errors during the transformation process, a new crossover and mutation method are proposed so that the CSD code structure still remains during the whole gene evolution process.

5.2. Gene crossover based on CSD structure

For the crossover with a CSD code structure, it is important that whether or not the CSD remained its structure after the crossover. Usually the crossover results as a non-CSD structure and a renewal of the code is required. In past research, the methods for solution were all to examine the structure after crossover. If a non-CSD structure was found, the code was renewed, where the code must be close to the numeral value of the original code. If the code structure was very lengthy, a longer time was required to inspect the code structure, and longer time spent on code renewal. This method causes over consumption of time. Therefore in the paper a method is proposed which does not spent a lengthy much of time to inspect the code structure, and does not require long times for code renewal. However the method in this paper must be operated only when the original parental gene is already a CSD structure, and thus the binary system code shall permanently maintain as a CSD structure. The following below are the principles for operation:

1. if $C_{N_1}^{P_1}$ & $C_{N_1-1}^{P_2} = 1$ then

$$C_{N_1-1}^{P_2} = \overline{C_{N_1-1}^{P_2}}$$

end

2. if $C_{N_2}^{P_1}$ & $C_{N_2+1}^{P_2} = 1$ then

$$C_{N_2}^{P_1} = \overline{C_{N_2}^{P_1}}$$

end

3. if $C_{N_1}^{P_2}$ & $C_{N_1-1}^{P_1} = 1$ then

$$C_{N_1-1}^{P_1} = \overline{C_{N_1-1}^{P_1}}$$

end

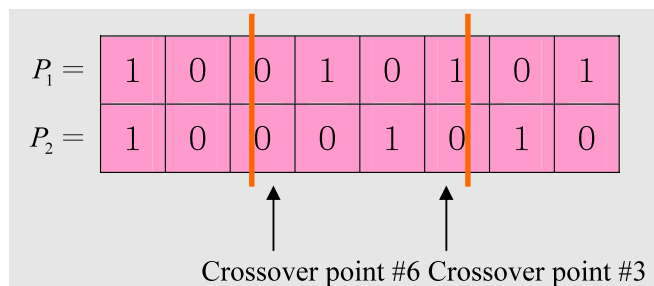
4. if $C_{N_2}^{P_2}$ & $C_{N_2+1}^{P_1} = 1$ then

$$C_{N_2}^{P_2} = \overline{C_{N_2}^{P_2}}$$

end

5. Proceed the crossover action of the two points P_1 and P_2 with N_1 and N_2 .

The notations $C_{N_i}^{P_i}$, $i = 1, 2$, represent the N_i th bit of the P_i th chromosome, and $\bar{C}$ represents the complement number of C . The symbol “&” represents the logical “and” operation.



For example, for the above two chromosomes in the parent generation P_1 and P_2 , the crossover is operated by using the 5 steps of above with the crossover points $N_1 = 3$, $N_2 = 6$, and the crossover results are as below.

1. $C_{N_1}^{P_1} \& C_{N_1-1}^{P_2} = 1$
 $\Rightarrow C_{N_1-1}^{P_2} = \overline{C_{N_1-1}^{P_2}} = 0.$
2. $C_{N_2}^{P_1} \& C_{N_2+1}^{P_2} = 0$
 $\Rightarrow$ unchanged.
3. $C_{N_1}^{P_2} \& C_{N_1-1}^{P_1} = 0$
 $\Rightarrow$ unchanged.
 $C_{N_2}^{P_2} \& C_{N_2+1}^{P_1} = 0$
 $\Rightarrow$ unchanged.

Perform crossover of the two points P_1 and P_2 with N_1 and N_2 respectively. The results are shown below:

Offspring $P_1 =$	1	0	0	0	1	0	0	1
Offspring $P_2 =$	1	0	0	1	0	1	0	0

After inspecting the transformation results which underwent through the five steps of above, it is apparently seen the CSD structure still was maintained, and the inspection of each bit is not required.

5.3. Gene mutation based on CSD structure

A general mutation simply changes the 0 to 1 or vice versa, 1 to 0. However for a CSD structure, the mutation will cause lost of the CSD structure. Thus a method is required so that the code can remain in the CSD structure after mutation. Otherwise, the effort to maintain CSD structure in the previous section by the crossover method would be invalid and redundant. Therefore this paper proposed a solution for the mutation, which does not use the method of past research.

The focus in this paper is only on the point of mutation and its two neighbor points. For a CSD structure only the value 1 would influence the CSD structure, thus only the CSD structure would be affected when a mutation transforms from 0 to 1, whereas the structure is not affected with the transform from 1 to 0. Thus when the value of the mutation point is 1, the CSD structure might be destroyed. As shown in the example below, P_1 is a CSD structure with 2 mutational points:

Before mutation,

$P_1 =$	0	1	0	1	0	1	0	0
---------	---	---	---	---	---	---	---	---

↑
point of mutation

After mutation,

Offspring $P_1 =$	0	1	0	1	0	1	1	0
-------------------	---	---	---	---	---	---	---	---

↑
non CSD structure

It can be seen from the example that the CSD structure disappeared after mutation. In order to maintain the existence of the CSD structure, the solution method from this paper is as follows:

Step 1. First of all inspect the numerical value of the point of mutation. As mentioned before, if the original value of the mutation point was 1, the structure will not be influenced after mutation. However if it was 0, the mutation transforming from 0 to 1 might cause the structure to change and cannot further follow the CSD structure. From the example of above, the value of mutation point 2 was 0. Thus after mutation a non-CSD structure might be created.

Step 2. The inspection of the mutation point transforming from 0 to 1 was already done in Step 1. Thus the next step is to inspect the values of the points adjacent to the mutational point. If the value of any of then is 1, suitable changes are required. If the adjacent values are both 0, no changes are required. In this example, the points adjacent to mutation point #2 are points #1 and #3, with values 0 and 1, respectively, in which after mutation, the CSD structure is definitely to be destructed.

Step 3. During the inspection in Step 2, we found that the CSD structure shall definitely vanish. Therefore to maintain the CSD structure in the paper, a change must be done for point #3, changing it from the original 1 to 0. Since in this example only point #3 has a value of 1, therefore the value is changed for only point #3. If the value of point #1 was also 1, point #1 would also need to change. Besides using the method in this paper, one can choose to not perform and mutation as method.

The solved mutation results are as shown below:

Offspring $P_1 =$	0	1	0	1	0	0	1	0
-------------------	---	---	---	---	---	---	---	---

From the results, it can be seen the CSD structure was maintained, and had also acquired mutational effect.

By following the three steps of above for operation, the genes will not form a non-CSD structure due to mutation. Therefore as long the operation is based with the method in this section, no matter after crossover or mutation, the structure still maintains and the process of re-inspection or code renewal are not required. With this method, the inspection time for each two-bit relations can be saved, and also saving the time of code renewal.

6. Simulation results

Assume the offset of the designed filter is set at $\delta_p = 0.05$ and $\delta_s = 0.001$, the transfer frequency band is 0.15π . By using the estimating equation mentioned in Section 2.2 to estimate the order of the filter, the filter order shall be $N_c = 27.024775$. Thus $N = 28$ is the order of the filter for our required design.

For the design of the low pass filter in the paper, the pass band width was 0.425π and the stop band width was 0.425π . The filter coefficient parameters are listed in Table 3. According with the procedures from the previous section, in this section the verification of the proposed method and efficiency in this paper was attempted on a symmetrical finite impulse digital low pass filter with 28 orders. In addition, the simulated results were also observed.

The mathematical parental number for GA in this paper was 40, bit length 270 (15 coefficients, where each coefficient was assembled from k_1, k_2, k_3 , with a bit length of 6 for each k_i). The crossover chance was 0.9, mutation chance was 0.01,

Table 3
Parameters of the filter coefficients.

Order	28
No. of coefficients k_i	3
Bit no. of every k_i	5
δ_p	0.05
δ_s	0.001
Pass band width	0.425π
Stop band width	0.425π
Transfer frequency band	0.15π

Table 4
GA parameters.

Mathematical parental number	40
No. of bits	270
Crossover chance	0.9
Mutation chance	0.01
Reserve quantity of better genes	4
Mathematical training number	100 000

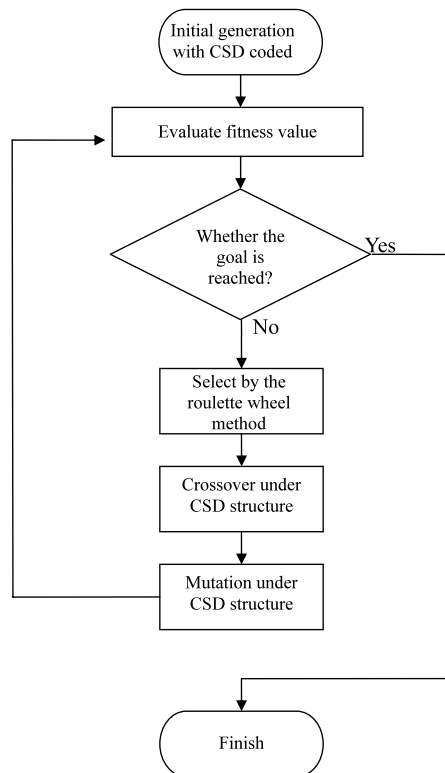


Fig. 5. Flow chart of GA framework.

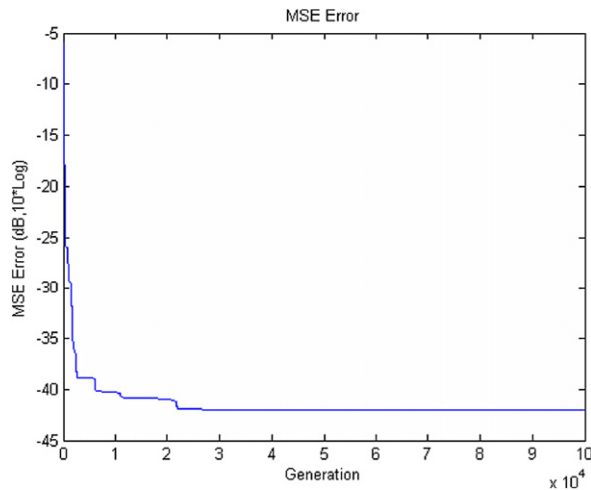


Fig. 6. GA convergence simulation based on CSD structure.

Table 5

Numerical data of the GA convergence based on CSD structure.

No. of generations	Mean square error	No. of generations	Mean square error
100	0.169926069135081	4000	0.000130310610534345
200	0.0377396821132693	5000	0.000129315464946452
300	0.00347155998172241	10 000	9.35633135984915e-005
400	0.00261887149888509	20 000	8.06307730288536e-005
500	0.00260161871871188	30 000	6.35793095232774e-005
1000	0.00251135715569537	40 000	6.35752031291314e-005
2000	0.000251245587124766	50 000	6.35731617230672e-005
3000	0.000130830139548433	100 000	6.34650359562179e-005

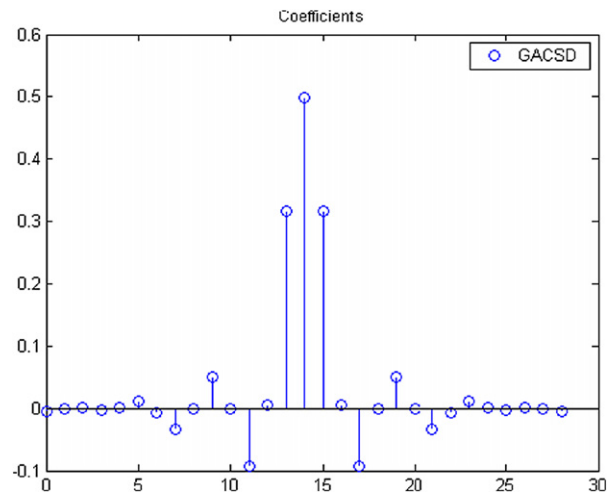


Fig. 7. Distribution of the CSD structure filter coefficients.

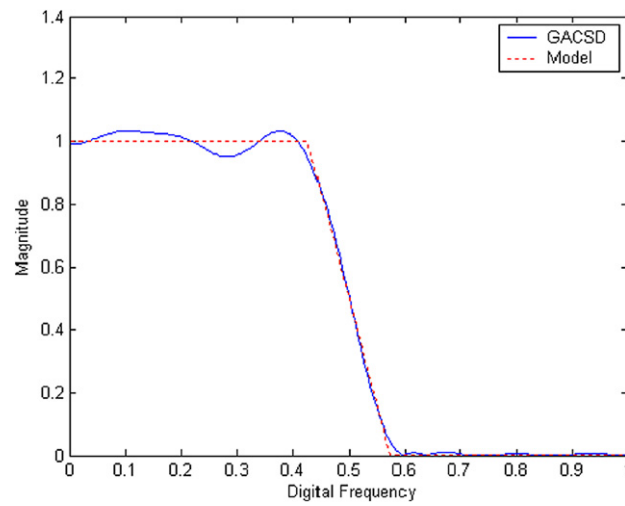


Fig. 8. CSD filter frequency response gain.

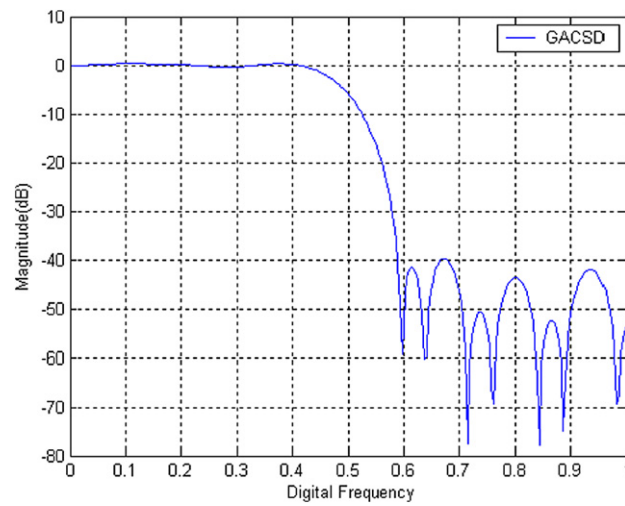


Fig. 9. CSD filter frequency response (dB).

Table 6
Filter coefficients of the CSD structure.

Coefficients	CSD code	Value
C(0)	000000 100000 101000	−0.0039063
C(1)	110000 000010 100000	−1.5259e−005
C(2)	001010 100010 000001	0.00085449
C(3)	101000 000100 000010	−0.0036011
C(4)	001001 001000 000000	0.0019684
C(5)	001000 000000 000000	0.011719
C(6)	101000 100000 101000	−0.0078278
C(7)	100100 000001 100100	−0.033203
C(8)	000100 100000 001000	0.00024414
C(9)	000101 000001 000010	0.050781
C(10)	110000 001010 010101	−1.5244e−005
C(11)	100100 100010 100000	−0.09375
C(12)	001000 000001 000010	0.0063477
C(13)	000010 000010 000100	0.31641
C(14)	000010 000000 101000	0.49902
C(15)	000010 000010 000100	0.31641
C(16)	001000 000001 000010	0.0063477
C(17)	100100 100010 100000	−0.09375
C(18)	110000 001010 010101	−1.5244e−005
C(19)	000101 000001 000010	0.050781
C(20)	000100 100000 001000	0.00024414
C(21)	100100 000001 100100	−0.033203
C(22)	101000 100000 101000	−0.0078278
C(23)	001000 000000 000000	0.011719
C(24)	001001 001000 000000	0.0019684
C(25)	101000 000100 000010	−0.0036011
C(26)	001010 100010 000001	0.00085449
C(27)	110000 000010 100000	−1.5259e−005
C(28)	000000 100000 101000	−0.0039063

and the reserve quantity for better genes was 4. The 4 groups of reserved genes were the best 4 among the groups, which the mathematical training number was 100 000 iterations. Table 4 lists the parameters for GA.

After undergoing the initial setup, the simulation flow chart is as shown in Fig. 5. First of all the initial generation must be the needed CSD code structure in this paper, which the initial generation was generated randomly. The initial value of the CSD structure was directly created from this randomly generation method. The afterward crossover and mutation only require the proposed method in this paper to maintain the CSD structure.

The corresponding simulated MSE convergence results obtained by the above method are listed in Table 5, and the MSE convergence plot is shown in Fig. 6. Simulated results of the CSD's filter coefficients distribution, the filter frequency response gain, and the filter frequency response (in dB) are shown in Figs. 7, 8, and 9, respectively. The final coefficients obtained in CSD structure are listed in Table 6.

7. Conclusions

The CSD code method used in this paper is more convenient for the direct realization on hardware application. The coding action is not required and can directly attain the simulation results onto the hardware, as where the realization of real-type algorithm on hardware must undergo coding, causing several possible errors and influence the final realized results. The newly proposed hybrid code method in this paper can raise the precision of the code. In addition, the paper also proposed a GA for the CSD code which does not destructs the filter coefficients, to still maintain the CSD code structure after each generation of chromosome evolution. The recursive try-and-error step is not required which can save quite much time. According to the final simulated results in the paper it was shown that the proposed algorithm in this paper had a quite good efficiency.

References

- [1] R.M. Hewlitt, E.S. Swartzlantz Jr., Canonical signed digit representation for FIR digital filters, in: IEEE Workshop on Signal Processing Systems, 11–13 Oct. 2000, pp. 416–426.
- [2] E. Cabal-Yepez, T.D. Carozzi, R. de J. Romero-Troncoso, M.P. Goughc, N. Huberc, FPGA-based system for frequency detection of the main periodic component in time series information, *Digital Signal Process.* 18 (2008) 1029–1044.
- [3] Z. Tang, J. Zhang, H. Min, A high-speed, programmable, CSD coefficient FIR filter, *IEEE Trans. Consumer Electron.* 48 (4) (2002) 834–837.
- [4] W. Gass, Automatic generation of high-performance ASICs, *Digital Signal Process.* 1 (1991) 15–16.
- [5] A.T.G. Fuller, B. Nowrouzian, F. Ashrafzadeh, A novel modified branch-and-bound technique for discrete optimization over canonical signed-digit number space, in: *Proceedings of the 1998 IEEE International Symposium on Circuits and Systems*, vol. 5, 31 May–3 June 1998, pp. 391–394.
- [6] A. Lee, M. Ahmadi, G.A. Jullien, W.C. Miller, R.S. Lashkari, Digital filter design using genetic algorithm, in: *IEEE Symposium on Advances in Digital Filtering and Signal Processing*, 5–6 June 1998, pp. 34–38.

- [7] F. Ashrafzadeh, B. Nowrouzian, Crossover and mutation in genetic algorithms employing canonical signed-digit number system, in: Proceedings of the 40th Midwest Symposium on Circuits and Systems, vol. 2, 3–6 Aug. 1997, pp. 702–705.
- [8] A.T.G. Fuller, B. Nowrouzian, F. Ashrafzadeh, Optimization of FIR digital filters over the canonical signed-digit coefficient space using genetic algorithms, in: Proceedings of the 1998 Midwest Symposium on Circuits and Systems, 9–12 Aug. 1998, pp. 456–459.
- [9] M. Oner, A genetic algorithm for optimisation of linear phase FIR filter coefficients, in: Conference Record of the 32nd Asilomar Conference on Signals, Systems & Computers, vol. 2, 1–4 Nov. 1998, pp. 1397–1400.
- [10] Q. Zhao, Y. Tadokoro, A simple design of FIR filters with powers-of-two coefficients, IEEE Trans. Circuits Syst. 35 (5) (1988) 566–570.
- [11] P. Gentili, F. Biazza, A. Uncini, Evolutionary design of FIR digital filters with power-of-two coefficients, in: Proceedings of the First IEEE Conference on Evolutionary Computation, IEEE World Congress on Computational Intelligence, vol. 1.1, 27–29 June 1994, pp. 110–114.
- [12] K.P. Estola, Designing power-of-two-Nth band FIR filters using Remez algorithm, in: ISCAS 1992 Proceedings of IEEE International Symposium on Circuits and Systems, vol. 2, 3–6 May 1992, pp. 569–572.
- [13] R.L. Haupt, S.E. Haupt, Practical Genetic Algorithms, Wiley, 1998.
- [14] C.C. Tseng, Minimax design of stable IIR digital filter with prescribed magnitude and phase responses, IEEE Trans. Circuits Syst. 1 49 (2002) 547–551.
- [15] I.C. Park, H.J. Kang, Digital filter synthesis based on an algorithm to generate all minimal signed digit representations, IEEE Trans. Computer-Aided Design Integrat. Circuits Syst. 21 (2002) 1525–1529.
- [16] K.Y. Khoo, A. Kwentus, A.N. Willson Jr., A programmable FIR digital filter using CSD coefficients, IEEE J. Solid-State Circuits 31 (1996) 869–874.
- [17] H.R. Lee, C.W. Jen, C.M. Liu, A new hardware-efficient architecture for programmable FIR filters, IEEE Trans. Circuits Syst. II: Analog and Digital Signal Processing 43 (1996) 637–644.
- [18] J.J. Grefenstette, Optimization of control parameters for genetic algorithm, IEEE Trans. Syst. Man Cybernet. 16 (1986) 122–128.
- [19] J.V. de Vegtea, Fundamentals of Digital Signal Processing, Prentice Hall, NJ, 2001.
- [20] E.C. Ifeachor, B.W. Jervis, Digital Signal Processing: A Practical Approach, Addison-Wesley, 1993.
- [21] R.L. Haupt, S.E. Haupt, Practical Genetic Algorithms, Wiley, New York, 1998.
- [22] L. Davis, Handbook of Genetic Algorithms, Van Nostrand/Reinhold, New York, 1991.
- [23] Chih-Hung Wu, Hung-Ju Chou, Wei-Han Su, A genetic approach for coordinate transformation test of GPS positioning, IEEE Geosci. Remote Sens. Lett. 4 (2) (2007) 297–301.
- [24] S.-H. Doong, Chih-Chin Lai, C.-H. Wu, Genetic subgradient method for solving location allocation problems, Appl. Soft Comput. 7 (2007) 373–386.
- [25] M. Wang, S. Yang, S. Wu, A GA-based UWB pulse waveform design method, Digital Signal Process. 18 (1) (2008) 65–74.
- [26] Y. Du, K.T. Chan, Feasibility of applying genetic algorithms in space-time block coded multiuser detection systems, Digital Signal Process. 15 (2005) 161–170.
- [27] S.T. Pan, C.C. Lai, Identification of chaotic systems by neural network with hybrid learning algorithm, Chaos Solitons Fractals 37 (1) (2008) 233–244.
- [28] C.C. Lai, C.Y. Chang, A hierarchical evolutionary algorithm for automatic medical image segmentation, Expert Syst. Appl. 36 (1) (2009) 248–259.

Shing-Tai Pan was born in Pingtung, Taiwan, in 1966. He received the M.S. degree in electrical engineering from National Sun Yat-Sen University, Kaohsiung, Taiwan, in 1992 and the Ph.D. degree from National Chiao Tung University, Hsinchu, Taiwan, in 1996. In 2006, he joined the Department of Computer Science and Information Engineering, National University of Kaohsiung, Kaohsiung, Taiwan, as an Associate Professor.

Shing-Tai Pan is a member of Taiwanese Association for Artificial Intelligence (TAAI) and Chinese Automatic Control Society (CACS). He is also a member of The Association for Computational Linguistics and Chinese Language Processing (ACLCLP). His current research interests are in the area of digital signal process, speech recognition, evolutionary computations, applications of artificial intelligent and intelligent control systems design.